

二分答案算法详解

导出时间：2026-05-28 10:54:37

二分答案算法详解

一、什么是二分答案？

二分答案是指在答案具有单调性的前提下，通过二分查找的方式在"答案区间"中寻找最优解。

与二分查找（在有序数组中找元素）不同：

- 二分查找：找的是已存在的元素
- 二分答案：找的是满足条件的最优值（最大/最小值）

二、什么时候用二分答案？

题目通常具有以下特征：

特征	说明
求最值	求"最大值最小"或"最小值最大"
答案单调	如果 x 满足条件，则 $x+1$ 也满足（或反之）
判定容易	给定一个值，容易判断它是否满足条件

常见题型关键词：

- "最大化的最小值"
- "最小化的最大值"
- "至少/至多"类问题
- 分配问题、切割问题、距离问题

三、核心思想

答案区间 [left, right]

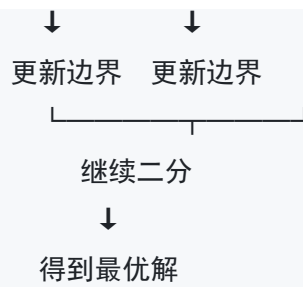


取中间值 mid



检查 mid 是否满足条件





关键：将"求最优解"转化为"判定问题"

四、算法模板

模板 1：求最小值的最大值（下界）

```
// 找满足条件的最小值（左边界）
int binary_search_left(int l, int r) {
    while (l < r) {
        int mid = l + (r - l) / 2; // 防溢出
        if (check(mid)) {
            // mid 满足条件，答案可能更小或就是 mid
            r = mid;
        } else {
            // mid 不满足，答案必须更大
            l = mid + 1;
        }
    }
    return l; // 或 r，此时 l == r
}
```

模板 2：求最大值的最大值（上界）

```
// 找满足条件的最大值（右边界）
int binary_search_right(int l, int r) {
    while (l < r) {
        int mid = l + (r - l + 1) / 2; // 注意 +1，避免死循环
        if (check(mid)) {
            // mid 满足条件，答案可能更大或就是 mid
            l = mid;
        } else {
            // mid 不满足，答案必须更小
            r = mid - 1;
        }
    }
}
```

```
    return l;
}
```

关键区别：mid 的计算方式

类型	mid 计算	移动方式	用途
找左边界	$l + (r-l)/2$	$r = mid$ 或 $l = mid + 1$	最小值最大化
找右边界	$l + (r-l+1)/2$	$l = mid$ 或 $r = mid - 1$	最大值最小化

为什么要+1?

- 当 $l = r - 1$ 时，若 $mid = l$ ，且 $check(mid)$ 为 true，则 $l = mid$ 会导致死循环
- +1 确保 mid 向上取整，避免这种情况

五、经典例题解析

例题 1：砍木头（NOIP 经典题）

问题：有 n 根木头，长度分别为 $L[i]$ ，需要砍出至少 k 段等长的木头，求每段木头的最大长度。

分析：

- 答案范围： $[1, \max(L[i])]$
- 单调性：如果能切出长度 x ，则一定能切出更短的长度
- 判定：给定长度 mid ，计算能切出多少段，判断 $\geq k$

```
bool check(int x) {
    if (x == 0) return true;
    long long cnt = 0;
    for (int i = 0; i < n; i++) {
        cnt += L[i] / x;
    }
    return cnt >= k;
}

int solve() {
    int l = 0, r = maxL; // 注意 l 可以从 0 开始
    while (l < r) {
        int mid = l + (r - l + 1) / 2;
        if (check(mid)) {
            l = mid; // 能满足，尝试更长
        } else {
            r = mid - 1;
        }
    }
}
```

```

    }
}
return l;
}

```

例题 2：Aggressive Cows (POJ 2456)

问题：n个牛栏排成一行，c头牛，每头牛放在一个牛栏，求任意两头牛之间的最小距离的最大值。

分析：

- 答案范围：[1, 最大坐标差]
- 判定：给定距离d，用贪心判断是否能让所有牛两两距离 $\geq d$

```

bool check(int d) {
    int cnt = 1; // 第一头牛放第一个位置
    int last = pos[0];
    for (int i = 1; i < n; i++) {
        if (pos[i] - last >= d) {
            cnt++;
            last = pos[i];
        }
    }
    return cnt >= c;
}

```

```

int solve() {
    sort(pos, pos + n);
    int l = 0, r = pos[n-1] - pos[0];
    while (l < r) {
        int mid = l + (r - l + 1) / 2;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}

```

六、与二分查找的对比

维度	二分查找	二分答案
查找对象	数组中的元素	满足条件的最优值

搜索空间	数组下标	答案的取值范围
核心操作	<code>arr[mid] == target</code>	<code>check(mid)</code>
返回值	元素下标或 -1	最优答案值
典型应用	查找元素	最优化问题

七、实战技巧与注意事项

1. 确定边界

```
// 左闭右闭区间 [l, r] 最常用
// 初始值要根据题意确定
int l = 1, r = 1e9; // 根据数据范围
```

2. 防止整数溢出

```
// 推荐写法
int mid = l + (r - l) / 2;
// 或 (C++20)
int mid = midpoint(l, r);
```

3. check 函数是核心

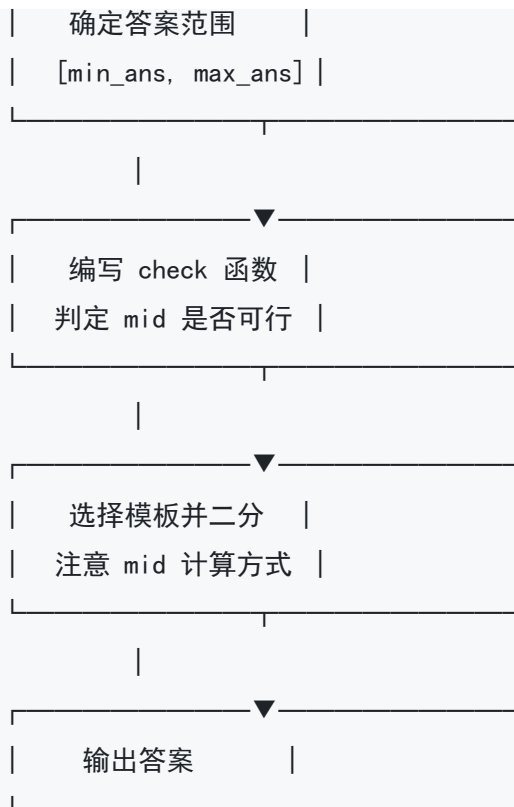
- 时间复杂度决定了整体效率
- 通常需要 $O(n)$ 或 $O(n \log n)$

4. 浮点数二分

```
// 实数域二分，控制精度
while (r - l > eps) { // eps 如 1e-6
    double mid = (l + r) / 2;
    if (check(mid)) l = mid;
    else r = mid;
}
```

八、总结流程图





核心要点：二分答案的精髓在于将最优化问题转化为判定问题，利用答案的单调性来高效求解。牢记两种模板（找左边界/右边界）的区别，特别是mid的计算方式，这是避免死循环的关键。